

Reminders and announcements

- ▶ During lectures you can submit questions to:
[vevox.app](https://vevox.com/join/107-018-952) ID: 107-018-952
- ▶ Any unanswered questions from Vevox will be answered on the COMP1870 Teams channel.
- ▶ Recordings of lectures (excluding the first lecture) are available through Minerva, in the *Panopto* section.

Definition

Definitions are fundamental in mathematics. It is critically important to memorise them.

- ▶ A **graph** consists of a set of **vertices** and a set of **edges**. Each edge joins a pair of vertices.
 - ▶ A vertex and an edge are **incident** if they touch.
 - ▶ A graph is **bipartite** if the vertices can be divided into two sets: X and Y , where every edge joins a vertex in X to a vertex in Y .
 - ▶ A **matching** in a graph is a set of some (maybe all) of the edges in the graph. If M is a matching, it must satisfy the rule:
 - ▶ every vertex is incident with at most one edge in M .
- It is equivalent to say:
- ▶ no vertex is incident with two or more edges in M .

Revision

- ▶ A vertex is **matched** if it is incident with one of the edges in M .
- ▶ A **path** is a sequence of vertices with no repetitions, such that any pair of consecutive vertices is joined by an edge.
- ▶ Say that M is a matching. A path P is **alternating** (relative to M) if the edges of P alternate between edges in M and edges not in M .

Augmenting paths

An alternating path is **augmenting** if the end vertices of the path are both unmatched.

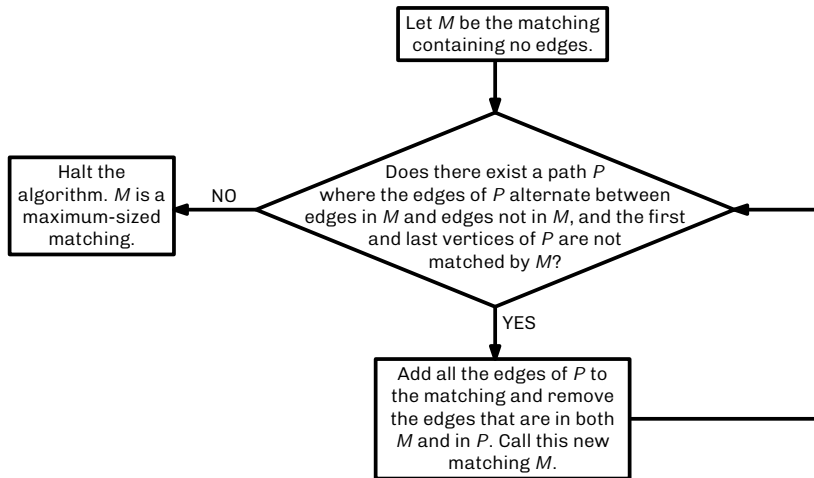
We can use an augmenting path to make a larger matching. Change the status of every edge in the path: edges in the matching are removed from the matching, edges not in the matching are added to the matching.

The augmenting paths algorithm

These ideas lead to an algorithm for finding a matching.

1. Start with the empty matching.
2. Look for an augmenting path.
3. If there is no augmenting path, then halt the algorithm.
4. If we find an augmenting path, use it to make a larger matching.
5. Go back to step 2.

The augmenting paths algorithm



The augmenting paths algorithm

Questions:

- ▶ Does this algorithm always find a matching of maximum size?
- ▶ If so, why?
- ▶ How do we look for augmenting paths?

The augmenting paths algorithm

Questions:

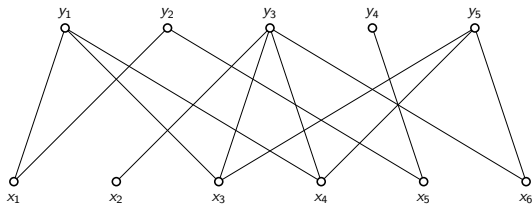
- ▶ Does this algorithm always find a matching of maximum size?

The answer is yes, because of the following fact.

Theorem 1.4. Let G be a bipartite graph and let M be a matching of G . If there is no augmenting path, then M is a maximum-sized matching.

The augmenting paths algorithm

We search for augmenting paths using a **breadth-first search**.



The breadth first search algorithm

Algorithm 1 The breadth-first search to find an augmenting path

Input: A bipartite graph, where every edge joins a vertex in the set X to a vertex in the set Y , and a matching M .

Output: An augmenting path P , or an empty output if there is no augmenting path.

```
1: Mark every vertex as unvisited.
2:  $U_X \leftarrow$  the set of vertices in  $X$  that are unmatched by  $M$ .
3:  $L \leftarrow$  the vertices in  $U_X$  listed in some order.
4: Mark the vertices in  $L$  as visited.
5: while  $L$  contains at least one vertex do
6:    $a \leftarrow$  the first vertex in  $L$ .
7:   Remove the first vertex from  $L$ .
8:   if  $a$  is in  $X$  then
9:     for unvisited vertex  $b$  that is joined to  $a$  by an edge do
10:      Mark  $b$  as visited.
11:       $\text{pred}(b) \leftarrow a$ 
12:      if  $b$  is matched by  $M$  then
13:        Add  $b$  to the end of list  $L$ . //Add  $b$  to the list for later processing.
14:      else  $b$  is unmatched by an edge in  $M$ . //We have found an augmenting path.
15:         $P \leftarrow$  a list containing the single vertex  $b$ 
16:         $c \leftarrow b$ 
17:        while  $c$  is not in  $U_X$  do //We build the path  $P$  by backtracking.
18:           $d \leftarrow \text{pred}(c)$ 
19:           $c \leftarrow d$ 
20:          Add  $c$  to the beginning of the list  $P$ .
21:        end while
22:        Halt and return  $P$ . //The path  $P$  is augmenting.
23:      end if
24:    end for
25:  else  $a$  is in  $Y$  //Note that in this case,  $a$  must be matched.
26:     $b \leftarrow$  the vertex in  $X$  that is joined to  $a$  by an edge in  $M$ .
27:    Mark  $b$  as visited.
28:     $\text{pred}(b) \leftarrow a$ .
29:    Add  $b$  to the end of list  $L$ .
30:  end if
31: end while //If the list becomes empty, there is no augmenting path.
```



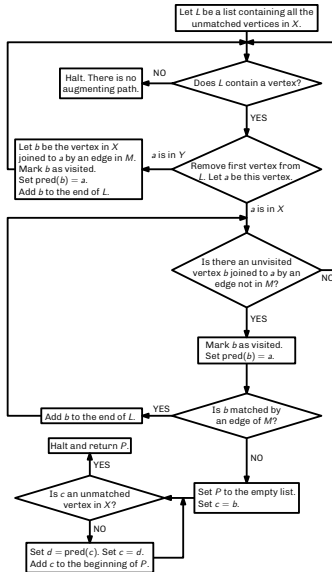
ab边不在匹配M中，且b是未访问节点

顶点B在匹配M中

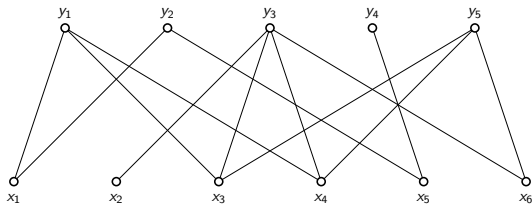
顶点B不在匹配 M中

a和X中的顶点组成的边在匹配M中

The breadth first search algorithm



Breadth-first search



Running time

The augmenting paths algorithm takes about $|V|^3$ steps to complete when the input is a graph with $|V|$ vertices.

This is much better than $2^{\frac{1}{4}|V|^2}$.

We aim for algorithms where the number of steps is something like n^c , where n is the size of the input and c is a (smallish) constant.

In this case, the algorithm runs in **polynomial time**.

Mathematics in computer science

How have we used mathematics?

- ▶ To develop the intuition which led to the idea of augmenting paths.
- ▶ To prove that if there is no augmenting path then the matching has maximum size (we haven't actually done this yet).
- ▶ To analyse the number of steps that algorithms use.
- ▶ To compare the running times of algorithms.

Logic

Logic originated in the study of arguments.

- ▶ What are we doing when we try to convince another person of something?
- ▶ What makes some arguments **valid** (convincing) and other arguments invalid?

Logic is studied in mathematics, linguistics, and philosophy.

Logic appears in computer science in multiple contexts, including:

- ▶ logical circuits,
- ▶ search queries,
- ▶ program verification.

Propositional logic

Logic is concerned with **formal languages** (languages with mathematically defined rules of grammar).

We start with the language of propositions. A **proposition** has one of two **truth values**: either true or false.

We typically denote propositions by $P_1, P_2, P_3, \dots$ (These are sometimes called **atomic propositions**.)

We will use **T** (true) and **F** (false) to denote truth values.

It is common to see truth values denoted by 1 (true) and 0 (false).

We use **operators** to combine statements into more complicated statements.

Logical operators

Negation (NOT):

A	$\neg A$
T	
F	

Logical operators

Conjunction (AND):

A	B	$A \wedge B$
T	T	
T	F	
F	T	
F	F	

Logical operators

Disjunction (OR):

A	B	$A \vee B$
T	T	
T	F	
F	T	
F	F	

Logical operators

Implication (IF THEN):

A	B	$A \Rightarrow B$
T	T	
T	F	
F	T	
F	F	

In a statement like $A \Rightarrow B$, we call A the **antecedent** and B the **consequent**.

Logical operators

If and only if (IFF):

A	B	$A \Leftrightarrow B$
T	T	
T	F	
F	T	
F	F	

Truth tables

We use operators to combine atomic propositions into more complicated statements, such as

$$(P_1 \vee P_2) \Rightarrow (P_2 \wedge (\neg P_3)).$$

We can describe the truth outputs from these statements in **truth tables**.

P_1	P_2	P_3	$(P_1 \vee P_2) \Rightarrow (P_2 \wedge (\neg P_3))$
T	T	T	
T	T	F	
T	F	T	
T	F	F	
F	T	T	
F	T	F	
F	F	T	
F	F	F	

Equivalence

Let A be a logical statement.

- ▶ A is **satisfiable** (or **consistent**) if A is true in at least one row of its truth table.
- ▶ A is **unsatisfiable** (or **inconsistent** or a **contradiction**) if it is false in every row of its truth table.
- ▶ A is a **tautology** if it is true in every row of its truth table.

Example. $A \wedge (\neg A)$ is a contradiction.

Example. $A \vee (\neg A)$ is a tautology.

Equivalence

Definition 2.3. Let A and B be statements. If $A \Leftrightarrow B$ is a tautology, then A and B are **equivalent**.

Proposition 2.5. The following statements are equivalent.

- ▶ $A \vee \mathbf{T}$ and $\mathbf{T}$
- ▶ $A \wedge \mathbf{T}$ and A
- ▶ $A \vee \mathbf{F}$ and A
- ▶ $A \wedge \mathbf{F}$ and $\mathbf{F}$

Equivalence

Proposition 2.6. The following statements are equivalent.

Double negation:	A and $\neg(\neg A)$
Commutative laws:	$A \wedge B$ and $B \wedge A$ $A \vee B$ and $B \vee A$
Associative laws:	$A \wedge (B \wedge C)$ and $(A \wedge B) \wedge C$ $A \vee (B \vee C)$ and $(A \vee B) \vee C$
Distributive laws:	$A \vee (B \wedge C)$ and $(A \vee B) \wedge (A \vee C)$ $A \wedge (B \vee C)$ and $(A \wedge B) \vee (A \wedge C)$
Idempotent laws:	$A \wedge A$ and A $A \vee A$ and A
De Morgan's laws:	$\neg(A \wedge B)$ and $(\neg A) \vee (\neg B)$ $\neg(A \vee B)$ and $(\neg A) \wedge (\neg B)$

Equivalence

Proposition 2.7. The following statements are equivalent.

- ▶ $A \Rightarrow B$ and $(\neg A) \vee B$
- ▶ $\neg(A \Rightarrow B)$ and $A \wedge (\neg B)$